

Day 02 - Git Workflow, Branching, Merging & Hands-On Practice

Objective

To gain practical experience with Git and GitHub by understanding the Git workflow, repository management, branching, merging, and version history.

Git Workflow

Git follows a three-stage workflow:

```
Working Directory → Staging Area → Repository
```

Working Directory

The area where files are created, modified, and deleted.

Staging Area

A temporary area where changes are prepared before committing.

Repository

The database where committed changes are permanently stored.

Repository Management

Creating a New Repository

```
git init
```

Initializes a new Git repository in the current directory.

Cloning an Existing Repository

```
git clone <repository-url>
```

Downloads a copy of a remote repository to the local machine.

Tracking File Changes

Check Repository Status

```
git status
```

Displays modified, staged, and untracked files.

Add Files to Staging Area

```
git add filename
```

Add all files:

```
git add .
```

Unstage Changes

```
git reset
```

Removes files from the staging area without deleting changes.

Creating Meaningful Commits

Create a Commit

```
git commit -m "Meaningful commit message"
```

Example:

```
git commit -m "Added project documentation"
```

Best Practices

- Write clear commit messages.
 - Keep commits focused on a single change.
 - Commit frequently.
-

Branching

Branches allow developers to work on features independently.

View Branches

```
git branch
```

Create a Branch

```
git branch feature-login
```

Switch Branches

```
git checkout feature-login
```

Create and Switch Simultaneously

```
git checkout -b feature-login
```

Merging Branches

Merging combines changes from one branch into another.

Merge Branch

```
git merge feature-login
```

Benefits:

- Combines completed work
 - Preserves development history
 - Enables team collaboration
-

Viewing Commit History

Show Commit History

```
git log
```

Provides:

- Commit ID
 - Author
 - Date
 - Commit message
-

Local vs Remote Repository

Local Repository

Stored on your computer.

Advantages:

- Faster access
- Offline work
- Personal development environment

Remote Repository

Hosted on GitHub.

Advantages:

- Backup of code
 - Team collaboration
 - Access from anywhere
-

Pushing Code to GitHub

Connect Repository

```
git remote add origin <repository-url>
```

Push Changes

```
git push origin main
```

Uploads local commits to GitHub.

Hands-On Practice Completed

- Created multiple repositories
- Added and modified files
- Deleted files and tracked changes
- Used staging and unstaging commands
- Created multiple commits
- Practiced branch creation
- Switched between branches
- Merged branches successfully

- Pushed repositories to GitHub
 - Explored version history
-

Commands Practiced Today

```
git init
git clone
git status
git add
git reset
git commit
git branch
git checkout
git merge
git log
git remote add origin
git push origin main
```

Key Takeaway

Git is not just a tool for saving code. It is a complete version control system that enables developers to experiment, collaborate, track changes, and recover from mistakes confidently. Every commit represents a step in the evolution of a project.

Day 02 Summary

Today focused on strengthening Git fundamentals through practical exercises. I learned how Git manages project history, how branches enable parallel development, and how repositories are synchronized with GitHub for collaboration and backup.

Status:  Day 02 Completed